

Emploid Scraping & Ingestion Pipeline

1. Scraping Architecture

Emploid relies on a Python pipeline to harvest and normalize active job listings from the web. The scrapers are located in the ``scripts/`` and ``scripts/scrapers/`` directories.

```
scripts/
run_scrapes.py      # Entry point coordinating scraper execution
run_scrapes.sh      # Shell wrapper executing scraper pipeline
requirements.txt     # Python package dependencies (supabase, requests)
ingest.py           # Data sync logic reading from stdin and writing to Supabase
scrapers/
  base.py           # Abstract base scraper class defining standards
  greenhouse.py     # greenhouse.io API scraper implementation
```

Base Scraper Interface (``scripts/scrapers/base.py``)

Defines the ``BaseScraper`` class:

- ``init(self, name)``: Initializes name.
- ``fetch(self)``: Fetch raw data from target API/website. Must be overridden.
- ``parse(self, raw_data)``: Parse raw data and return a list of normalized job dictionaries. Must be overridden.
- ``run(self)``: High-level execution flow that runs ``fetch``, then ``parse``, and outputs each normalized job dictionary as a single-line JSON string to standard output (``stdout``).

Greenhouse Scraper (``scripts/scrapers/greenhouse.py``)

Scrapes active jobs from companies using Greenhouse ATS.

- **Method:** Greenhouse provides a public JSON API endpoint: ``https://boards-api.greenhouse.io/v1/boards/{board_token}/jobs?content=true``.
- **Flow:**
 - Iterates over a list of pre-configured company tokens (e.g. Vercel, Retool, Stripe, Linear, Dropbox).
 - Fetches the active listings and extracts: ``title``, ``location``, ``description``, ``apply_url`` (Greenhouse job page), ``posted_at``, and company metadata.
 - Normalizes remote/hybrid categories by checking locations for substrings like ``"Remote"`` or ``"Hybrid"``.

2. Ingestion Engine (`scripts/ingest.py`)

The ingestion script is a pipeline utility that runs inside a python process, reading a stream of normalized JSON job objects from standard input (`stdin`) and upserting them into the Supabase database.

Core Stages of `ingest.py`:

Stage 1: Stdin Parsing Reads lines from `sys.stdin` one-by-one. Each non-empty line is parsed as JSON.

Stage 2: Company Resolution For each job object, it resolves the associated company: 1. Extracts `company` name string (e.g. "Linear"). 2. Generates a URL-safe slug using a regex-based `slugify` helper (normalizes accents, replaces symbols, strips characters, maps `&` to `and`, collapses multiple hyphens). 3. Executes a Supabase UPSERT on `public.companies` targeting the `slug` column. 4. Retrieves the generated company ID (`comp\_id`) to associate with the job insertion.

Stage 3: Validation Validates that required fields `title` and `apply\_url` exist. If missing, it prints a warning to `stderr` and skips the listing.

Stage 4: Deduplication Strategy To avoid duplicate jobs from multiple aggregations, the script runs a two-tiered deduplication check:

- **Tier 1 (Source URL):** Queries `public.jobs` for rows matching `source\_url`. If found, this is a direct match.
- **Tier 2 (Composite Key):** If `source\_url` isn't found, it queries `public.jobs` matching `company\_id` + `title` + `source` (+ `location` if present).

Stage 5: Database Upsert

- **If match found:** Performs a target row `UPDATE`. It overwrites fields that are non-empty in the incoming scraper data, and updates `last\_seen\_at` to the current timestamp.
- **If match not found:** Performs an `INSERT`. It creates a new record in `public.jobs`, setting `first\_seen\_at` and `last\_seen\_at` to the current time, and initializing `repost\_count = 0`.

3. Database Aggregation Triggers

When jobs are updated, inserted, or deleted in the `jobs` table, company statistics must stay in sync. This is handled by a database trigger.

Trigger Logic (`update\_company\_stats\_trg`)

1. **Trigger Definition:** Configured as `AFTER INSERT OR UPDATE OR DELETE ON public.jobs FOR EACH ROW`. 2. **Stat Recalculation:** The trigger function executes a deferred UPDATE on `public.companies` for the associated `company\_id`:

- Sets `total\_active\_listings` to the count of jobs with `is\_active = true` for that company.
- Sets `avg\_ghost\_score` to the SQL average of the `ghost\_score` of all active jobs for that company.

3. **Bulk Performance Optimization:** The trigger checks if the ``ghost_score`` or ``is_active`` fields actually changed. If they are unchanged during bulk updates (e.g. daily scraper sweeps simply updating ``last_seen_at``), the recalculation queries are bypassed, avoiding database overhead.